Hiring Data Analytics Project Report

1. Project Description:

a.) Project Objective –

The primary objective of this project is to showcase core technical competencies in Python data processing and build an automated pipeline bridging Python with relational database systems. Beyond isolated scripting, the project demonstrates how to connect Python to a MySQL database using SQLAlchemy and PyMySQL. This unified pipeline enables business stakeholders to run advanced analytical SQL queries—leveraging Common Table Expressions (CTEs), window functions, and multi-tier aggregations—to evaluate organizational hiring performance and workforce payroll commitments efficiently.

b.) Project Overview –

The Hiring Data Analytics project is an end-to-end data analysis project which is focused on evaluating recruitment throughput, department hiring rates, and compensation trends across 7,000+ applicant records. The workflow begins with systematic data cleaning in Python using Pandas, resolving real-world data discrepancies such as converting Excel serial timestamps into standardized dates, handling missing values through domain-specific imputation, stripping inconsistent text whitespace, and pruning statistical outliers using the Interquartile Range (IQR) technique. Following preparation, comprehensive exploratory data analysis (EDA) and visualization were performed using Matplotlib and Seaborn to identify critical operational trends, including applicant concentration across departments, gender-based selection rates, salary dispersion, and monthly interview volume patterns.

2. Dataset Overview:

Columns Names –

application_id, interview_taken_on, status, gender, department, post_name, offered_salary

No. of Rows – **7168**

No. of Columns – **7**

Data Source – **Trainity.**

3. Tech Stack Used:

Python (Jupyter Notebook) – It is used to run scripts.

Python (Pandas) – It is used to clean dataset and performing Exploratory Data Analytics (EDA).

Python (Matplotlib & Seaborn) – It is used to do EDA through Visualizing data.

MySQL – Used for further data analysis.

4. Data Cleaning:

Before embarking on any analysis, it is essential to ensure the dataset is clean and reliable. The data cleaning process involves handling missing values, correcting inconsistencies, and formatting the data for analysis. In this project, we carefully cleaned the dataset to ensure the accuracy and integrity of our findings.

1. Importing dataset into Python –

[1]:
```
import numpy as np
import pandas as pd
```

[2]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

[3]:
```
df = pd.read_csv(r"C:\Users\Pradeep\Downloads\statistics_csv_4.csv")
```

[4]:
```
df.head()
```

[4]:

	application_id	Interview Taken on	Status	event_name	Department	Post Name	Offered Salary
0	383422	41760.48668	Hired	Male	Service Department	c8	56553.0
1	907518	41765.33926	Hired	Female	Service Department	c5	22075.0
2	176719	41765.33951	Rejected	Male	Service Department	c5	70069.0
3	429799	41761.68679	Rejected	Female	Operations Department	i4	3207.0
4	253651	41761.68919	Hired	Male	Operations Department	i4	29668.0

[5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7168 entries, 0 to 7167
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   application_id      7168 non-null   int64
 1   Interview Taken on  7168 non-null   float64
 2   Status              7168 non-null   str
 3   event_name          7168 non-null   str
 4   Department          7164 non-null   str
 5   Post Name           7168 non-null   str
 6   Offered Salary      7167 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 392.1 KB
```

2. Formatting the column names –

[6]:
```
df.columns = df.columns.str.lower()
```

[7]:
```
df.head(2)
```

[7]:

	application_id	interview taken on	status	event_name	department	post name	offered salary
0	383422	41760.48668	Hired	Male	Service Department	c8	56553.0
1	907518	41765.33926	Hired	Female	Service Department	c5	22075.0

[8]:
```
df.columns= df.columns.str.replace(' ','_')
```

[9]:
```
df.head(2)
```

[9]:

	application_id	interview_taken_on	status	event_name	department	post_name	offered_salary
0	383422	41760.48668	Hired	Male	Service Department	c8	56553.0
1	907518	41765.33926	Hired	Female	Service Department	c5	22075.0

```
[10]: df = df.rename(columns = {'event_name' : 'gender'})
```

```
[11]: df.head(2)
```

[11]:

	application_id	interview_taken_on	status	gender	department	post_name	offered_salary
0	383422	41760.48668	Hired	Male	Service Department	c8	56553.0
1	907518	41765.33926	Hired	Female	Service Department	c5	22075.0

3. Checking and Removing duplicates –

```
[12]: print('duplicate rows',df.duplicated().sum())
```

```
duplicate rows 0
```

```
[13]: print(df.duplicated(subset=['application_id']).sum())
```

```
27
```

```
[14]: df = df.drop_duplicates(subset= ['application_id'])
```

```
[15]: df.info()
```

```
<class 'pandas.DataFrame'>
Index: 7141 entries, 0 to 7167
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   application_id      7141 non-null   int64  
 1   interview_taken_on  7141 non-null   float64
 2   status              7141 non-null   str    
 3   gender              7141 non-null   str    
 4   department          7137 non-null   str    
 5   post_name           7141 non-null   str    
 6   offered_salary      7140 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 446.3 KB
```

4. Handling Null Values –

```
[16]: df = df.dropna(subset=['department']).reset_index(drop =True)
```

```
[17]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7137 entries, 0 to 7136
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   application_id      7137 non-null   int64  
 1   interview_taken_on  7137 non-null   float64
 2   status              7137 non-null   str    
 3   gender              7137 non-null   str    
 4   department          7137 non-null   str    
 5   post_name           7137 non-null   str    
 6   offered_salary      7136 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 390.4 KB
```

4.1 Replacing the null value in the offered_salary column with its department average.

```
df.loc[df['offered_salary'].isna(), : ]
```

	application_id	interview_taken_on	status	gender	department	post_name	offered_salary
77	114584	41766.33896	Rejected	Male	Sales Department	i7	NaN

```
df['offered_salary'] = df.groupby('department')['offered_salary'].transform(lambda x: x.fillna(x.mean()))
```

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7137 entries, 0 to 7136
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   application_id      7137 non-null   int64  
 1   interview_taken_on  7137 non-null   float64
 2   status              7137 non-null   str    
 3   gender              7137 non-null   str    
 4   department          7137 non-null   str    
 5   post_name           7137 non-null   str    
 6   offered_salary      7137 non-null   float64
dtypes: float64(2), int64(1), str(4)
memory usage: 390.4 KB
```

5. Changing the column value and its type to datetime –

```
df['interview_taken_on'] = pd.to_datetime(df['interview_taken_on'], unit='D', origin='1899-12-30')
```

```
df['interview_taken_on'] = df['interview_taken_on'].dt.floor('D')
```

```
df.head(3)
```

	application_id	interview_taken_on	status	gender	department	post_name	offered_salary
0	383422	2014-05-01	Hired	Male	Service Department	c8	56553.0
1	907518	2014-05-06	Hired	Female	Service Department	c5	22075.0
2	176719	2014-05-06	Rejected	Male	Service Department	c5	70069.0

6. Checking values in each column for correctness and accuracy –

```
df['status'].unique()
```

```
<StringArray>
['Hired', 'Rejected']
Length: 2, dtype: str
```

```
df['gender'].unique()
```

```
<StringArray>
['Male', 'Female', '-', 'Don’t want to say']
Length: 4, dtype: str
```

```
df['gender'] = df['gender'].str.replace('-','Don’t want to say')
```

```
df['gender'].unique()
```

```
<StringArray>
['Male', 'Female', 'Don’t want to say']
Length: 3, dtype: str
```

```
df['department'].unique()
```

```
<StringArray>
[       'Service Department',     'Operations Department',
          'Sales Department',        'Finance Department',
     'Production Department',       'Purchase Department',
      'Marketing Department',        'General Management',
 'Human Resource Department']
Length: 9, dtype: str
```

```
df['department'] = df['department'].str.replace('Department', '', case=False).str.strip()
# case = case sensitivity
```

```
df['department'].unique()
```

```
<StringArray>
[           'Service',         'Operations',              'Sales',
           'Finance',         'Production',           'Purchase',
         'Marketing', 'General Management',     'Human Resource']
Length: 9, dtype: str
```

[31]:
```
df['post_name'].unique()
```

[31]:
```
<StringArray>
[  'c8',   'c5',   'i4',    '-',   'i7',  'n10',   'b9',   'i5',   'i1',
   'i6',   'm6',   'm7', 'c-10',   'c9',   'n9',   'n6']
Length: 16, dtype: str
```

[32]:
```
df['post_name'] = df['post_name'].str.replace('-','')
```

[33]:
```
df['post_name'].unique()
```

[33]:
```
<StringArray>
[ 'c8',  'c5',  'i4',    '',  'i7', 'n10',  'b9',  'i5',  'i1',  'i6',  'm6',
  'm7', 'c10',  'c9',  'n9',  'n6']
Length: 16, dtype: str
```

```
df['post_name'] = df['post_name'].replace('','Not Mentioned')
```

```
df['post_name'].unique()
```

```
<StringArray>
[           'c8',            'c5',            'i4', 'Not Mentioned',
            'i7',           'n10',            'b9',            'i5',
            'i1',            'i6',            'm6',            'm7',
           'c10',            'c9',            'n9',            'n6']
Length: 16, dtype: str
```

7. Dataset information after cleaning it –

```
[37]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7137 entries, 0 to 7136
Data columns (total 7 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   application_id      7137 non-null   int64
 1   interview_taken_on  7137 non-null   datetime64[ns]
 2   status              7137 non-null   str
 3   gender              7137 non-null   str
 4   department          7137 non-null   str
 5   post_name           7137 non-null   str
 6   offered_salary      7137 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(1), str(4)
memory usage: 390.4 KB
```

Initially the dataset has 7167 values but after removing duplicates and fixing null values, only 7137 values left. The datatype of date column has also been fixed. And the column names and its values had also been formatted and cleaned.

Now, the dataset is clean and we can use it for further Analysis.

5. Exploratory Data Analytics (EDA):

1. checking the basic statistical summary of data –

```
[8]: df.describe(include ='all')
```

[8]:

	application_id	interview_taken_on	status	gender	department	post_name	offered_salary
count	7137.000000	7137	7137	7137	7137	7137	7137.000000
unique	NaN	NaN	2	3	9	16	NaN
top	NaN	NaN	Hired	Male	Operations	c9	NaN
freq	NaN	NaN	4679	4069	2760	1783	NaN
mean	508208.095698	2014-07-02 08:29:03.253467648	NaN	NaN	NaN	NaN	50015.006550
min	12073.000000	2014-05-01 00:00:00	NaN	NaN	NaN	NaN	100.000000
25%	261521.000000	2014-06-02 00:00:00	NaN	NaN	NaN	NaN	25583.000000
50%	507460.000000	2014-07-04 00:00:00	NaN	NaN	NaN	NaN	49658.000000
75%	756858.000000	2014-08-01 00:00:00	NaN	NaN	NaN	NaN	74463.000000
max	999936.000000	2014-08-31 00:00:00	NaN	NaN	NaN	NaN	400000.000000
std	285145.839623	NaN	NaN	NaN	NaN	NaN	28852.003259

2. checking for outliers in the offered_salary column –

```
[39]: q1 = df['offered_salary'].quantile(0.25)
      q3 = df['offered_salary'].quantile(0.75)
      IQR = q3 - q1
```

```
[40]: lower_bound = q1 - 1.5*IQR
      upper_bound = q3 + 1.5*IQR
```

```
[41]: outliers = df[(df['offered_salary']<lower_bound) | (df['offered_salary']>upper_bound)]
```

```
[42]: outliers.shape
```

```
[42]: (3, 7)
```

There are 3 outliers in the dataset. In next step we will remove the outliers to maintain the data accuracy.

```
[43]: df = df[(df['offered_salary'] >= lower_bound) & (df['offered_salary'] <= upper_bound)].reset_index(drop=True)
```

```
[44]: df.describe()
```

[44]:

	application_id	interview_taken_on	offered_salary
count	7134.000000	7134	7134.000000
mean	508096.781890	2014-07-02 08:40:10.092514816	49909.882499
min	12073.000000	2014-05-01 00:00:00	100.000000
25%	261260.750000	2014-06-02 00:00:00	25582.250000
50%	507291.500000	2014-07-04 00:00:00	49633.500000
75%	756613.500000	2014-08-01 00:00:00	74418.500000
max	999936.000000	2014-08-31 00:00:00	99967.000000
std	285147.681025	NaN	28349.429586

All the outliers were removed.

3. Checking the hiring conversion rate by each department.

```
62]: print("\n=== HIRING CONVERSION BY DEPARTMENT ===\n")
dept_conversion = df.groupby('department')['status'].value_counts( normalize = True).unstack() * 100
print(dept_conversion.round(2))
```

```
=== HIRING CONVERSION BY DEPARTMENT ===

status              Hired  Rejected
department
Finance             61.32     38.68
General Management  66.07     33.93
Human Resource      71.88     28.12
Marketing           62.15     37.85
Operations          66.52     33.48
Production          64.81     35.19
Purchase            69.28     30.72
Sales               64.92     35.08
Service             64.77     35.23
```

Top Selection Rate: Human Resource achieved the highest interview success rate at 71.88%, closely followed by Purchase at 69.28%.

Selective Screening: Finance recorded the strictest screening criteria with a 61.32% acceptance rate and a 38.68% rejection rate.

4. Checking the salary breakdown by department –

```
[53]: print("\n=== SALARY BREAKDOWN BY DEPARTMENT ===\n")
salary_by_dept = df.groupby('department')['offered_salary'].agg(['count', 'mean', 'median', 'std']).round(2)
print(salary_by_dept)
```

```
=== SALARY BREAKDOWN BY DEPARTMENT ===

                  count      mean   median       std
department
Finance             287  49759.30  51497.0  27583.67
General Management  168  55327.04  58709.5  27696.54
Human Resource       96  49448.64  52374.0  29919.49
Marketing           325  48489.94  45843.0  29392.53
Operations         2760  49142.23  47804.5  28442.38
Production          378  49434.38  46704.5  28644.07
Purchase            332  52669.64  55193.0  27992.69
Sales               744  49403.75  48844.5  27927.44
Service            2044  50593.68  50430.0  28254.20
```

General Management exhibits the highest average compensation (55327.04).

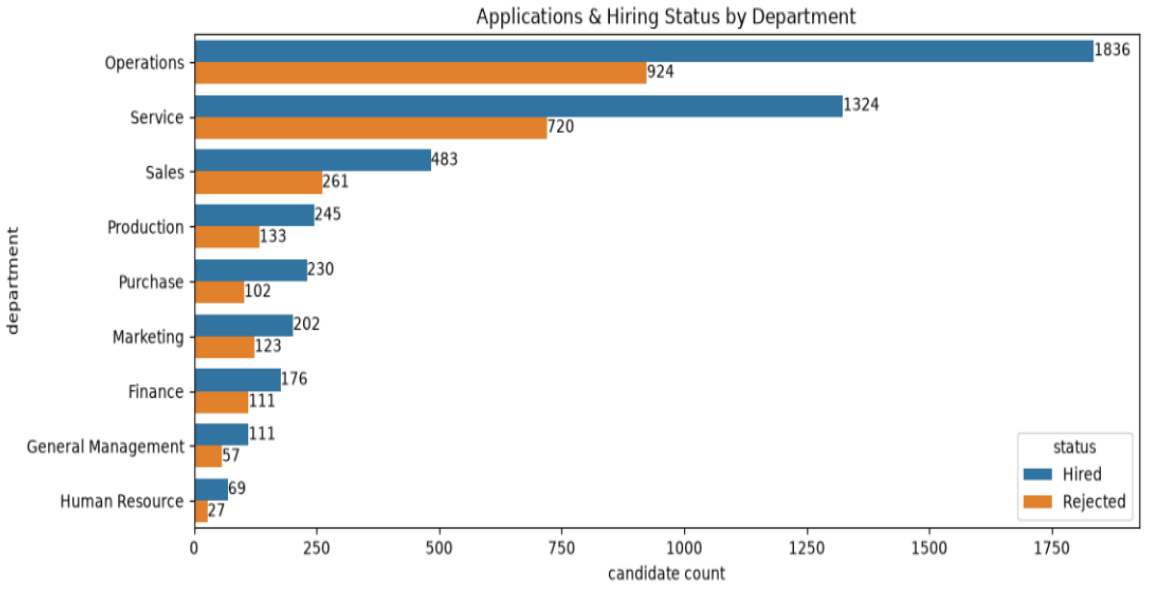
5. Checking Applications & Hiring Volume by Department using Bar Chart –

```
[52]: plt.figure(figsize=(12,5))

dept_order = df['department'].value_counts().index
ax = sns.countplot(data=df,y='department',hue = 'status',order = dept_order)
plt.xlabel('candidate count')
plt.ylabel('department')
plt.legend(title = 'status')
plt.title('Applications & Hiring Status by Department')

for bars in ax.containers:
    ax.bar_label(bars)

plt.show()
```



Application Dominance: Operations (2,760 applicants) and Service (2,045 applicants) together account for over 67% of total hiring activity.

Hiring Output: Operations successfully onboarded the largest workforce contingent (1,836 hires), demonstrating where company operational capacity is expanding most rapidly.

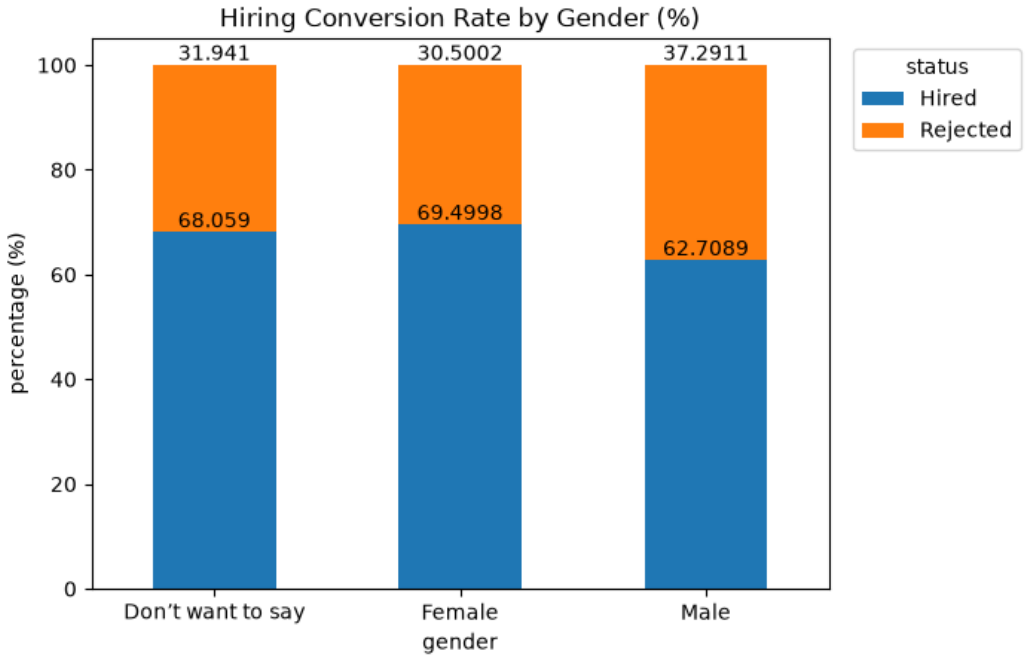
6. checking Hiring Conversion Rate by Gender using Stacked Bar Chart –

[58]:
```
gender_hire = pd.crosstab(df['gender'], df['status'], normalize='index') * 100
ay = gender_hire.plot(kind = 'bar',stacked=True,rot=0)

plt.xlabel('gender')
plt.ylabel('percentage (%)')
plt.legend(title = 'status',bbox_to_anchor=(1.02, 1))
plt.title('Hiring Conversion Rate by Gender (%)')

for bars in ay.containers:
    ay.bar_label(bars)

plt.show()
```



The applicant pool shows heavy male representation, with males accounting for 57.01% (4,069) and females making up 37.28% (2,661).

Female candidates converted at a higher success rate (69.52%) than male applicants (62.72%), a gap of nearly 6.8 percentage points

7. What percentage of all hired & rejected candidates were female vs. male.

[126]:

status	Hired	Rejected
gender		
Don't want to say	5.923867	5.288853
Female	39.520958	32.994304
Male	54.555175	61.716843

Among the total rejected candidates, male candidates have comparatively higher chance of being rejected in comparison to female candidates.

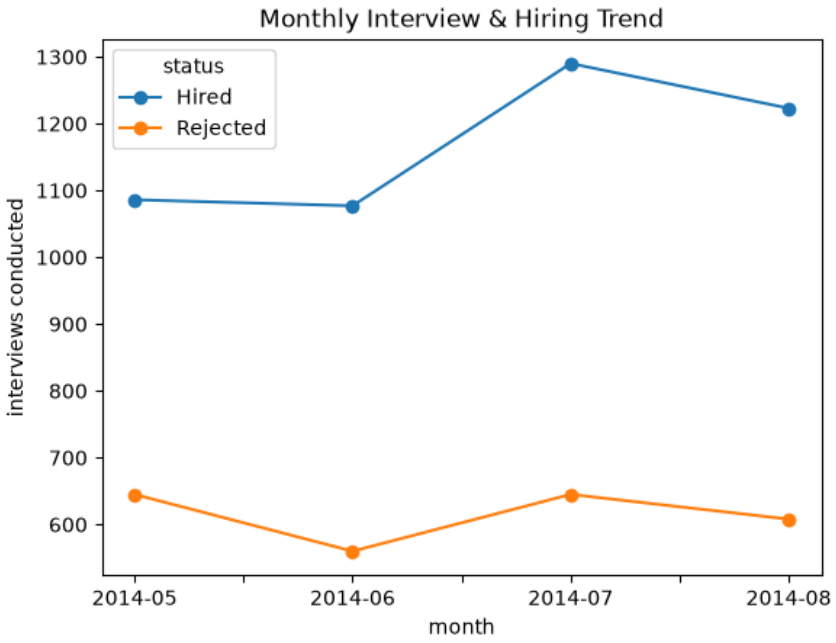
8. Checking Monthly Recruitment & Interview Trend using Line Chart –

[63]:
```
df['interview_month_year'] = df['interview_taken_on'].dt.strftime('%Y-%m')
monthly_trend = df[['interview_month_year', 'status']].value_counts().unstack()
monthly_trend.plot(kind='line',marker='o')

plt.xlabel('month')
plt.ylabel('interviews conducted')
plt.title('Monthly Interview & Hiring Trend')

plt.show
```

[63]:
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



Interview volume remained consistently active above 1,600 evaluations per month, reaching an operational peak in July 2014 with 1,936 interviews.

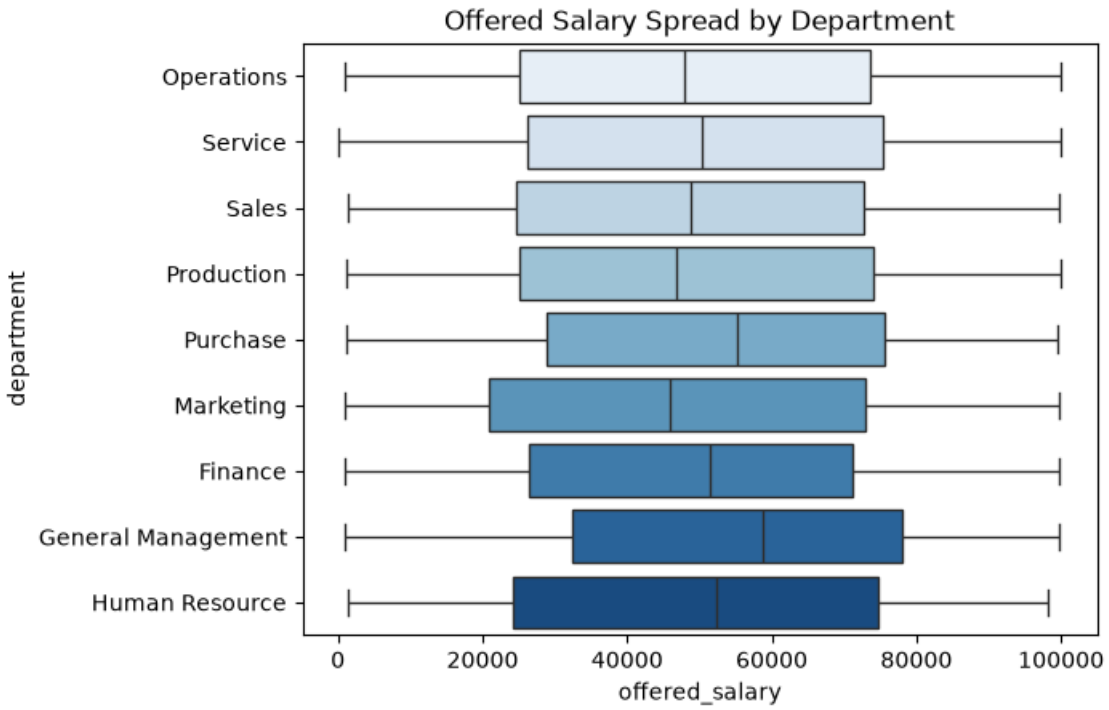
The ratio of hired to rejected candidates stayed remarkably stable throughout the 4-month span, averaging approximately 2 hired candidates for every 1 rejected.

Following July's peak, activity stabilized at 1,831 interviews in August, indicating a structured hiring surge mid-summer

9. Checking Salary Distribution by Department using Boxplot –

```
[90]: dept_order = df['department'].value_counts().index
      sns.boxplot(x='offered_salary',y = 'department',data = df,order = dept_order,palette='Blues')
      plt.xlabel('offered_salary')
      plt.ylabel('department')
      plt.title('Offered Salary Spread by Department')

      plt.show()
```



Across seven of the nine departments, median offered salaries sit consistently between 46,000 and 55,000.

General Management exhibits the widest dispersion and highest average compensation.

6. Data Analysis using MySQL:

In order to perform the data analysis, first step is to export the data from Python to MySQL database. For this we will write a script in python to directly export the cleaned dataset to MySQL database.

```
[131]: %pip install pymysql sqlalchemy
```

```
[133]: from sqlalchemy import create_engine

# MySQL connection
username = "root"
password = "9829829628Kk"
host = "localhost"
port = "3306"
database = "hiring_data"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")

# Write DataFrame to MySQL
table_name = "candidates"   # choose any table name
df.to_sql(table_name, engine, if_exists="replace", index=False)
```

```
[133]: 7134
```

The entire dataset is loaded in the MySQL database and ready to use.

```
select * from candidates;
```

application_id	interview_taken_on	status	gender	department	post_name	offered_salary
383422	2014-05-01	Hired	Male	Service	c8	56553
907518	2014-05-06	Hired	Female	Service	c5	22075
176719	2014-05-06	Rejected	Male	Service	c5	70069
429799	2014-05-02	Rejected	Female	Operations	i4	3207
253651	2014-05-02	Hired	Male	Operations	i4	29668
289907	2014-05-01	Hired	Male	Sales	Not Mentioned	85914
959124	2014-05-06	Rejected	Male	Sales	i7	69904
86642	2014-05-09	Rejected	Male	Sales	i7	11758
751029	2014-05-02	Hired	Female	Service	i4	15156
434547	2014-05-02	Rejected	Female	Service	i4	49515
518854	2014-05-01	Rejected	Male	Service	n10	26990
199526	2014-05-07	Hired	Male	Service	b9	86787
539803	2014-05-15	Hired	Male	Finance	b9	2308
191009	2014-05-09	Hired	Female	Service	i7	56688

candidates 1 ×

1. How many candidates fall into each tier, and what is the average salary within each tier?

```sql
SELECT
    CASE
        WHEN offered_salary < 30000 THEN 'Tier 1 (< 30k)'
        WHEN offered_salary BETWEEN 30000 AND 60000 THEN 'Tier 2 (30k - 60k)'
        ELSE 'Tier 3 (> 60k)'
    END AS salary_tier,
    COUNT(*) AS total_candidates,
    ROUND(AVG(offered_salary), 2) AS avg_salary
FROM candidates
GROUP BY salary_tier
ORDER BY avg_salary ASC;
```

salary_tier	total_candidates	avg_salary
Tier 1 (< 30k)	2108	15351.09
Tier 2 (30k - 60k)	2235	45179.77
Tier 3 (> 60k)	2791	79799.43

The largest portion of applicants falls into Tier 3 (> 60k) with 2,794 candidates (39.15%), averaging an offered salary of 79799.43.

The tiered distribution demonstrates a multi-level hiring structure, maintaining dedicated pay bands for entry apprentices, middle specialists, and senior technical leads.

2. Which departments have an average offered salary strictly greater than the overall company-wide average offered salary?

```sql
SELECT
    department,
    ROUND(AVG(offered_salary), 2) AS dept_avg_salary
FROM candidates
GROUP BY department
HAVING AVG(offered_salary) > (
    SELECT AVG(offered_salary)
    FROM candidates
)
ORDER BY dept_avg_salary DESC;
```

department	dept_avg_salary
General Management	55327.04
Purchase	52669.64
Service	50593.68

Only 3 out of 9 departments—General Management (55327.04), Purchase (52,669.64), and Service (50593.68)—exceed the company-wide baseline.

These three units represent the primary cost drivers per head, requiring targeted budget allocation during annual recruitment planning.

3. For candidates who were successfully Hired, what is the total annualized financial commitment (SUM) and head count for each department?

```sql
SELECT
    department,
    COUNT(*) AS hired_count,
    ROUND(SUM(offered_salary), 2) AS total_payroll_expense,
    ROUND(AVG(offered_salary), 2) AS avg_hired_salary
FROM candidates
WHERE status = 'Hired'
GROUP BY department
ORDER BY total_payroll_expense DESC;
```

department	hired_count	total_payroll_expense	avg_hired_salary
Operations	1836	89795659	48908.31
Service	1324	66802175	50454.81
Sales	483	23474308	48601.05
Production	245	12117348	49458.56
Purchase	230	11979912	52086.57
Marketing	202	9664366	47843.4
Finance	176	8579698	48748.28
General Management	111	6171553	55599.58
Human Resource	69	3424856	49635.59

Operations and Service command the lion's share of financial commitment, absorbing 89.80M (38.4%) and 67.00M (28.7%) of total hired payroll respectively.

4. Which top 5 job post designations (post_name) brought in the highest number of hired candidates?

```sql
SELECT
    post_name,
    COUNT(*) AS hired_count,
    ROUND(AVG(offered_salary), 2) AS avg_offered_salary
FROM candidates
WHERE status = 'Hired'
GROUP BY post_name
ORDER BY hired_count DESC
LIMIT 5;
```

post_name	hired_count	avg_offered_salary
c9	1234	50382.83
c5	1179	49527.82
i7	631	50111.21
i5	510	49551.15
i6	337	47685.54

Job roles c9 (1,234 hires) and c5 (1,179 hires) represent the company's core hiring engines, making up 51.57% of all successful placements.

5. On which days of the week were the most interviews scheduled, and how does candidate throughput compare between weekdays and weekends?

```sql
SELECT
    DAYNAME(interview_taken_on) AS day_of_week,
    COUNT(*) AS total_interviews,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM candidates), 2) AS pct_of_total_interviews
FROM candidates
GROUP BY DAYNAME(interview_taken_on)
order by pct_of_total_interviews ;
```

day_of_week	total_interviews	pct_of_total_interviews
Sunday	278	3.90
Saturday	530	7.43
Monday	1070	15.00
Tuesday	1210	16.96
Wednesday	1260	17.66
Thursday	1318	18.47
Friday	1468	20.58

Interview activity peaks towards the end of the working week, with Friday recording the highest volume at 1,468 interviews (20.57%), followed by Thursday at 1,318 (18.47%).

Despite being standard non-working days, 809 interviews (11.34%) were logged on Saturdays and Sundays, highlighting overtime recruitment drives or shift-based hiring rounds.

6. How many candidates offered between 40,000 and 60,000 were marked as 'Rejected' and 'hired', grouped by department?

```
with cte as (select * from (SELECT
    department ,
    COUNT(*) AS rejected_mid_tier_count
FROM candidates
WHERE status = 'Rejected'
  AND offered_salary BETWEEN 40000 AND 60000
GROUP BY department) t
join
(SELECT
    department as dpt,
    COUNT(*) AS hired_mid_tier_count
FROM candidates
WHERE status = 'hired'
  AND offered_salary BETWEEN 40000 AND 60000
GROUP BY department) k
on t.department=k.dpt)
select department,rejected_mid_tier_count,hired_mid_tier_count
from cte;
```

department	rejected_mid_tier_count	hired_mid_tier_count
Service	144	286
Operations	209	394
Purchase	17	56
Finance	24	45
Marketing	27	42
Sales	55	103
Production	22	60
Human Resource	2	12
General Management	11	22

This high volume of rejected mid-tier candidates reveals potential mismatches between candidate qualification benchmarks and role expectations, signalling an opportunity to refine early-stage resume screening.

7. Conclusion:

The **Hiring Data Analytics** project demonstrates a complete, production-grade analytics workflow—spanning programmatic data cleaning, statistical discovery, exploratory visualizations, and relational database querying. By transforming raw, unformatted recruitment records into a standardized dataset, the analysis eliminated data integrity issues including duplicate entries, missing department salaries, malformed text, and extreme compensation outliers. The exploratory findings highlight an organization heavily scaling its frontline capacity, where Operations and Service represent over two-thirds of total recruitment volume and account for nearly 67% of total payroll commitment. Across 7,137 evaluated candidates, the recruitment funnel maintained a 65.56% hiring conversion rate with predictable salary bands across middle-tier roles, while leadership bands drove compensation variance.

Technically, the project establishes a robust data bridge between Python and MySQL. Leveraging Pandas, Matplotlib, and Seaborn delivered automated statistical profiling and clear visual narratives, while the transition to MySQL via SQLAlchemy showcased efficient relational modeling, schema typing, and advanced SQL querying using CTEs, window functions, and multi-tier aggregations. Ultimately, this project illustrates how end-to-end data pipelines turn unstructured operational hiring logs into high-impact business intelligence, equipping leadership with the quantitative insights needed to optimize talent acquisition, streamline departmental budgets, and balance workforce allocation.

END.